Human_Segmentation

April 1, 2026

1 Khoroshilova Elizaveta

[29]:
```
import os
import random
from pathlib import Path
import json
import shutil
import zipfile
from urllib.error import URLError
from urllib.request import urlretrieve
import cv2
import matplotlib_inline
import matplotlib.pyplot as plt
import numpy as np
import numpy.testing as npt
import torch
import torch.nn.functional as F
import torchvision.transforms.functional as TF
from PIL import Image, ImageDraw
from skimage import measure
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from torchvision.models import vgg13, VGG13_Weights
from tqdm import tqdm

%matplotlib inline
matplotlib_inline.backend_inline.set_matplotlib_formats('pdf', 'svg')
```

init initializing dataset objects, saves paths to images and masks **len** PyTorch needs to know how many total items are in your dataset so it knows when an epoch is finished

getitem can search for specific image, read it, use colour palette, read masks, apply transformations

[3]:
```
class PhotosDataset(Dataset):
    def __init__(self, images_dir, target_dir=None, transforms=None):
        """Load image (and optional mask) paths for segmentation.

        Args:
```

```
            images_dir (str): Directory of source images.
            target_dir (str | None): Directory of mask images (.png), or None␣
↪if unused.
            transforms (Callable | None): Optional transform applied as␣
↪``transforms(image, mask)``.

        Returns:
            None
        """
        self.images_dir = images_dir
        self.target_dir = target_dir
        self.transforms = transforms
        self.image_filenames = sorted(os.listdir(images_dir))

    def __len__(self):
        """Return the number of images in the dataset.

        Args:
            None

        Returns:
            int: Dataset length.
        """
        return len(self.image_filenames)

    def __getitem__(self, idx):
        """Load one sample by index.

        Args:
            idx (int): Index in ``[0, len(self))``.

        Returns:
            tuple: ``(image, mask)`` where ``mask`` may be ``None`` if␣
↪``target_dir`` is None.
        """
        img_name = self.image_filenames[idx]
        img_path = os.path.join(self.images_dir, img_name)

        image = cv2.imread(img_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        mask = None
        if self.target_dir is not None:
            mask_name = img_name.split('.')[0] + '.png'
            mask_path = os.path.join(self.target_dir, mask_name)
            mask = cv2.imread(mask_path, 0)
```

```
        image = Image.fromarray(image)
        if mask is not None:
            mask = Image.fromarray(mask)

        if self.transforms is not None:
            if mask is not None:
                image, mask = self.transforms(image, mask)
            else:
                image, _ = self.transforms(image, None)

        return image, mask
```

The constructor of the standard `ImageFolder' takes the`transform`parameter (and sometimes`target_transform‘).

They are used to convert uploaded images (usually `PIL.Image`) or targets into tensors of the desired type. Torchvision includes the transformations module for standard examples of such transformations. Transformations can contain random transformations, which is the easiest way to implement data augmentation. When defining a custom transformer, in addition to the constructor, you only need to implement the `__call__` method.:

```
class HorizontalFlip(object):
    def __init__(self, mode=0):
        self.method = mode

    def __call__(self, img):
        """
        Args:
            img (PIL.Image): Image to be flipped.

        Returns:
            PIL.Image: Randomly flipped image.
        """
        if self.method:
            return img.transpose(Image.FLIP_LEFT_RIGHT)
        return img
```

[4]:
```
class ComposePair:
    """Chain transforms that operate on (image, mask) pairs."""

    def __init__(self, transforms_list):
        """Build a composition of pair transforms.

        Args:
            transforms_list (list): Ordered callables, each ``(image, mask) ->␣
↪(image, mask)``.

        Returns:
```

```python
            None
        """
        self.transforms_list = transforms_list

    def __call__(self, image, mask):
        """Apply each transform in sequence.

        Args:
            image (PIL.Image.Image): RGB image.
            mask (PIL.Image.Image | None): Label mask, or None.

        Returns:
            tuple: ``(image, mask)`` after all transforms.
        """
        for t in self.transforms_list:
            image, mask = t(image, mask)
        return image, mask

class RandomFlipPair:
    """Random horizontal flip for image and mask."""

    def __init__(self, p=0.5):
        """Configure horizontal flip probability.

        Args:
            p (float): Probability of applying the flip, in ``[0, 1]``.

        Returns:
            None
        """
        self.p = p

    def __call__(self, image, mask):
        """Apply a random horizontal flip to the image and mask.

        Args:
            image (PIL.Image.Image): RGB image.
            mask (PIL.Image.Image): Label mask.

        Returns:
            tuple: ``(image, mask)``, possibly flipped.
        """
        if random.random() < self.p:
            image = TF.hflip(image)
            mask = TF.hflip(mask)
        return image, mask
```

```python
class RandomCutoutPair:
    """Random rectangular cutout on image and mask."""

    def __init__(self, p=0.5, size=50):
        """Configure random square cutout.

        Args:
            p (float): Probability of applying cutout.
            size (int): Square side length in pixels.

        Returns:
            None
        """
        self.p = p
        self.size = size

    def __call__(self, image, mask):
        """Draw a random black square on the image and mask.

        Args:
            image (PIL.Image.Image): RGB image.
            mask (PIL.Image.Image): Label mask.

        Returns:
            tuple: ``(image, mask)`` with optional black rectangle.
        """
        if random.random() < self.p:
            w, h = image.size
            x0 = random.randint(0, w - self.size)
            y0 = random.randint(0, h - self.size)
            x1 = x0 + self.size
            y1 = y0 + self.size
            ImageDraw.Draw(image).rectangle([x0, y0, x1, y1], fill=(0, 0, 0))
            ImageDraw.Draw(mask).rectangle([x0, y0, x1, y1], fill=0)
        return image, mask

class RandomBrightnessPair:
    """Random multiplicative brightness on the image only."""

    def __init__(self, p=0.5, limit=0.2):
        """Configure random brightness jitter.

        Args:
            p (float): Probability of changing brightness.
            limit (float): Max relative deviation from 1.0 (symmetric).

        Returns:
```

```
            None
        """
        self.p = p
        self.limit = limit

    def __call__(self, image, mask):
        """Adjust image brightness; leave mask unchanged.

        Args:
            image (PIL.Image.Image): RGB image.
            mask (PIL.Image.Image): Label mask (unchanged).

        Returns:
            tuple: ``(image, mask)``.
        """
        if random.random() < self.p:
            factor = 1.0 + random.uniform(-self.limit, self.limit)
            image = TF.adjust_brightness(image, factor)
        return image, mask

class RandomBackgroundPair:
    """Composite foreground from mask over a random background image."""

    def __init__(self, p=0.5, bg_dir=None):
        """Load optional background image paths for compositing.

        Args:
            p (float): Probability of swapping background.
            bg_dir (str | None): Folder of background images; if missing, no-op.

        Returns:
            None
        """
        self.p = p
        self.bg_paths = []
        if bg_dir and os.path.exists(bg_dir):
            self.bg_paths = [os.path.join(bg_dir, f) for f in os.
↪listdir(bg_dir) if f.endswith(('.jpg', '.png', '.jpeg'))]

    def __call__(self, image, mask):
        """Composite foreground over a random background using the mask.

        Args:
            image (PIL.Image.Image): RGB image.
            mask (PIL.Image.Image): Label mask.

        Returns:
```

```
            tuple: ``(image, mask)`` with optional new background.
        """
        if random.random() < self.p and len(self.bg_paths) > 0:
            bg_path = random.choice(self.bg_paths)
            bg = Image.open(bg_path).convert("RGB")
            bg = bg.resize(image.size)
            binary_mask = mask.point(lambda p: 255 if p > 0 else 0, mode="1")
            image = Image.composite(image, bg, binary_mask)
        return image, mask

class ToTensorAndNormalizePair:
    """Convert image to ImageNet-normalized tensor; mask to float tensor."""

    def __init__(self):
        """Use ImageNet mean/std for images; float tensor masks scaled to [0,␣
↪1].

        Args:
            None

        Returns:
            None
        """
        self.to_tensor = transforms.ToTensor()
        self.normalize = transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )

    def __call__(self, image, mask):
        """Args:
            image (PIL.Image.Image): RGB image.
            mask (PIL.Image.Image | None): Label mask, or None.

        Returns:
            tuple: ``(image_tensor, mask_tensor)`` with shapes ``[C,H,W]`` and␣
↪``[1,H,W]`` (mask).
        """
        image = self.to_tensor(image)
        image = self.normalize(image)

        mask_tensor = torch.as_tensor(np.array(mask), dtype=torch.float32)
        if len(mask_tensor.shape) == 2:
            mask_tensor = mask_tensor.unsqueeze(0)

        if mask_tensor.max() > 1:
            mask_tensor = mask_tensor / 255.0
```

```
        return image, mask_tensor
```

This how we will add random transformations

```
transforms.ToTensor()
transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
```

[5]:
```
# Dataset location
#
# 1. Local folder (default): if `people_data/train` and `people_data/
↪train_mask` exist under the
#    current working directory or under `human_segmentation/`, that folder is␣
↪used (no download).
#
# 2. Public URL: otherwise the next cell downloads a **zip** once and caches␣
↪`people_data/` next
#    to the notebook:
#
#    Get from: https://github.com/l-horosh/human_segmentation/releases/download/
↪dataset-v1/people_data.zip
```

[6]:
```
# Public zip default (override with env PEOPLE_DATA_ZIP_URL).
_DEFAULT_PEOPLE_DATA_ZIP = (
    "https://github.com/l-horosh/human_segmentation/releases/download/
↪dataset-v1/people_data.zip"
)


def load_people_data_and_build_datasets():
    """Resolve data root, download zip if needed, and build train/test datasets.

    Args:
        None

    Returns:
        tuple[str, PhotosDataset, PhotosDataset]: ``DATA_PATH``, training set,␣
↪test set.
    """
    people_data_zip_url = os.environ.get("PEOPLE_DATA_ZIP_URL",␣
↪_DEFAULT_PEOPLE_DATA_ZIP)
    _cwd = Path.cwd()
    _search_roots = [_cwd, _cwd / "human_segmentation"]
    data_path = None
    for _base in _search_roots:
```

```
        _pd = _base.resolve() / "people_data"
        if _pd.is_dir() and (_pd / "train").is_dir() and (_pd / "train_mask").
↪is_dir():
            data_path = str(_pd)
            print(f"Using local dataset at {data_path}")
            break

    if data_path is None:
        _url = (people_data_zip_url or "").strip()
        if not _url:
            raise ValueError(
                "No local people_data/ with train/ and train_mask/. Set␣
↪PEPLE_DATA_ZIP_URL "
                "to a public zip URL."
            )
        _cache = _search_roots[0].resolve()
        _final = _cache / "people_data"
        if not (_final.is_dir() and (_final / "train").is_dir() and (_final /␣
↪"train_mask").is_dir()):
            _cache.mkdir(parents=True, exist_ok=True)
            _zip = _cache / "people_data.zip"
            print(f"Downloading dataset from {_url!r} ...")
            try:
                urlretrieve(_url, _zip)
            except URLError as e:
                raise RuntimeError(f"Failed to download dataset: {e}") from e
            if not zipfile.is_zipfile(_zip):
                _zip.unlink(missing_ok=True)
                raise RuntimeError("Downloaded file is not a valid zip archive.
↪")
            _stage = _cache / "_people_data_staging"
            if _stage.exists():
                shutil.rmtree(_stage)
            _stage.mkdir(parents=True, exist_ok=True)
            shutil.unpack_archive(_zip, _stage)

            def _find_root(d: Path):
                """Find extracted folder that contains ``train/`` and␣
↪``train_mask/``.

                Args:
                    d (pathlib.Path): Staging directory after unzip.

                Returns:
                    pathlib.Path | None: Root with train/mask dirs, or None.
                """
                d = d.resolve()
```

```
                if (d / "train").is_dir() and (d / "train_mask").is_dir():
                    return d
                n = d / "people_data"
                if (n / "train").is_dir() and (n / "train_mask").is_dir():
                    return n
                return None

            _found = _find_root(_stage)
            if _found is None:
                shutil.rmtree(_stage, ignore_errors=True)
                _zip.unlink(missing_ok=True)
                raise RuntimeError(
                    "Zip must contain train/ and train_mask/ at root or under␣
↪people_data/."
                )
            if _final.exists():
                shutil.rmtree(_final)
            shutil.move(str(_found), str(_final))
            shutil.rmtree(_stage, ignore_errors=True)
            _zip.unlink(missing_ok=True)
        data_path = str(_final)

    train_transforms = ComposePair([
        RandomFlipPair(p=0.5),
        RandomCutoutPair(p=0.3, size=60),
        RandomBrightnessPair(p=0.5, limit=0.2),
        RandomBackgroundPair(p=0.3, bg_dir=os.path.join(data_path,␣
↪"backgrounds")),
        ToTensorAndNormalizePair(),
    ])
    test_transforms = ComposePair([ToTensorAndNormalizePair()])
    train_ds = PhotosDataset(
        images_dir=os.path.join(data_path, "train"),
        target_dir=os.path.join(data_path, "train_mask"),
        transforms=train_transforms,
    )
    test_ds = PhotosDataset(
        images_dir=os.path.join(data_path, "test"),
        target_dir=os.path.join(data_path, "test_mask"),
        transforms=test_transforms,
    )
    return data_path, train_ds, test_ds


DATA_PATH, train_dataset, test_dataset = load_people_data_and_build_datasets()
```

```
Using local dataset at C:\Users\saf\Documents\gbc\t431\4015_adv-
```

math\project\human_segmentation\people_data

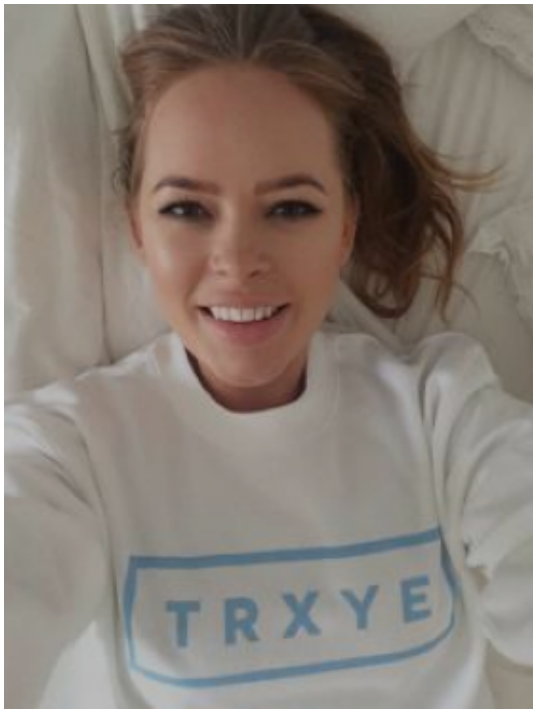
For displaying image:

[7]:
```
def show_idx_image(dataset, idx):
    """Denormalize and plot one dataset sample (image + mask).

    Args:
        dataset (torch.utils.data.Dataset): Dataset yielding tensor image and␣
↪mask.
        idx (int): Sample index.

    Returns:
        None
    """
    image, mask = dataset[idx]

    image = image.permute(1, 2, 0).numpy()
    image = (image * np.array([0.229, 0.224, 0.225])) + np.array([0.485, 0.456,␣
↪0.406])
    image = np.clip(image, 0, 1)

    mask = mask.squeeze(0).numpy()

    fig, axes = plt.subplots(1, 2, figsize=(6, 4))

    axes[0].imshow(image)
    axes[1].imshow(mask)

    axes[0].set_axis_off()
    axes[1].set_axis_off()

    fig.tight_layout()
    plt.show()
```

[8]:
```
def run_augmentation_smoke_tests(data_path: str, idx_to_show: int = 0):
    """Visualize each augmentation pipeline on one training sample.

    Args:
        data_path (str): Root ``people_data`` directory.
        idx_to_show (int): Dataset index to render.

    Returns:
        None
    """
    test_modules = {
        "1. Flip": ComposePair([RandomFlipPair(p=1.0),␣
↪ToTensorAndNormalizePair()]),
```

```
        "2. Cutout": ComposePair([RandomCutoutPair(p=1.0, size=80),␣
↪ToTensorAndNormalizePair()]),
        "3. Brightness": ComposePair([RandomBrightnessPair(p=1.0, limit=0.4),␣
↪ToTensorAndNormalizePair()]),
        "4. Background": ComposePair(
            [RandomBackgroundPair(p=1.0, bg_dir="path/to/backgrounds"),␣
↪ToTensorAndNormalizePair()]
        ),
    }
    for _name, transform in test_modules.items():
        temp_dataset = PhotosDataset(
            images_dir=os.path.join(data_path, "train"),
            target_dir=os.path.join(data_path, "train_mask"),
            transforms=transform,
        )
        show_idx_image(temp_dataset, idx_to_show)


run_augmentation_smoke_tests(DATA_PATH, idx_to_show=0)
```

TRXYE


TRXYE

When learning the network, it is more convenient to work with uploaders rather than datasets. Loaders are created based on a dataset and allow you to iterate over batches from it.

Pay attention to the `DataLoader` parameters. With `num_workers` > 1, the batches are being prepared (images are uploaded, processed, and so on) in several background processes at once. Using the `shuffle' parameter, you can submit images for training in random order. Usually, the data processing speed on the GPU exceeds the data loading speed in a single process, so the typical value for`num_workers‘ is≈ 3-10.

[9]:
```
def build_preview_data_loaders(
    train_dataset,
    test_dataset,
    batch_size: int = 8,
    num_workers: int = 3,
):
    """Construct DataLoaders with fixed worker count (preview / exploration).

    Args:
        train_dataset (Dataset): Training dataset.
        test_dataset (Dataset): Test dataset.
        batch_size (int): Batch size.
        num_workers (int): ``DataLoader`` workers.

    Returns:
```

```
        tuple[DataLoader, DataLoader]: ``(test_data_loader,␣
↪train_data_loader)``.
    """
    test_data_loader = DataLoader(test_dataset, batch_size=batch_size,␣
↪num_workers=num_workers)
    train_data_loader = DataLoader(
        train_dataset, batch_size=batch_size, num_workers=num_workers,␣
↪shuffle=True
    )
    return test_data_loader, train_data_loader


test_data_loader, train_data_loader = build_preview_data_loaders(train_dataset,␣
↪test_dataset)
```

The loss function is implemented like all standard neural modules in pytorch, via the `torch.nn.Module'. In the cell below, you are asked to implement the loss functiondice‘` (an analog of the Jacquard measure).

The result of **Dice Loss** is defined as:

$$\text{Dice Coefficient} = 2\frac{|A \cap B|}{|A| + |B|} \rightsquigarrow \text{Dice Loss} = 1 - \frac{2\sum\limits_{i,j} a_{ij} b_{ij}}{\sum\limits_{ij}(a_{ij} + b_{ij} + \varepsilon)}$$

where $a_{ij} \in [0, 1]$ is the predicted probability of finding a person in an image pixel, and b_{ij} is the true markup for an image pixel.

[10]:
```
class DiceLoss(torch.nn.Module):
    def __init__(self, eps=1e-7, reduction=None, with_logits=True):
        """Dice loss for binary segmentation.

        Args:
            eps (float): Small constant added to the denominator for stability.
            reduction (str | None): ``'mean'``, ``'sum'``, or ``None``␣
↪(per-sample vector).
            with_logits (bool): If True, apply sigmoid to ``logits`` before␣
↪Dice.

        Returns:
            None
        """
        super().__init__()
        self.eps = eps
        self.reduction = reduction
        self.with_logits = with_logits
```

```
    def forward(self, logits, true_labels):
        """Compute Dice loss between predictions and labels.

        Args:
            logits (torch.Tensor): Shape ``[B, ...]``; logits or probabilities␣
↪per ``with_logits``.
            true_labels (torch.Tensor): Binary (or integer) mask, same spatial␣
↪shape as logits.

        Returns:
            torch.Tensor: Scalar if ``reduction`` is ``'mean'`` or ``'sum'``,␣
↪else shape ``[B]``.
        """
        true_labels = true_labels.to(torch.long)

        if self.with_logits:
            logits = torch.sigmoid(logits)

        logits_flat = logits.view(logits.size(0), -1)
        true_labels_flat = true_labels.view(true_labels.size(0), -1).float()

        intersection = torch.sum(logits_flat * true_labels_flat, dim=1)

        denominator = torch.sum(logits_flat + true_labels_flat, dim=1) + self.
↪eps

        loss_value = 1.0 - (2.0 * intersection) / denominator

        if self.reduction == 'sum':
            loss_value = loss_value.sum()
        elif self.reduction == 'mean':
            loss_value = loss_value.mean()
        elif self.reduction is None:
            pass

        return loss_value
```

Checking

[11]:
```
def run_dice_loss_unit_tests():
    """Assert ``DiceLoss`` reductions match reference values.

    Args:
        None

    Returns:
        None
```

```
    """
    logits = torch.tensor(
        [
            [[0, 0.5], [0.5, 1]],
            [[0.1, 0.1], [0, 0]],
        ]
    )
    target = torch.tensor(
        [
            [[0, 1], [1, 1]],
            [[1, 0], [0, 1]],
        ]
    )
    losses = DiceLoss(with_logits=False, reduction=None, eps=1e-7)(logits,␣
↪target)
    npt.assert_almost_equal(losses.numpy(), np.array([0.2, 0.90909]), decimal=4)
    loss = DiceLoss(with_logits=False, reduction="mean", eps=1e-7)(logits,␣
↪target)
    npt.assert_almost_equal(float(loss.numpy()), 0.554545, decimal=4)
    loss = DiceLoss(with_logits=False, reduction="sum", eps=1e-7)(logits,␣
↪target)
    npt.assert_almost_equal(float(loss.numpy()), 1.10909, decimal=4)


run_dice_loss_unit_tests()
```

U-net schema

Blocks of VGG13 looks like:

```
    (0): Conv2d(...)
    (1): ReLU(...)
    (2): Conv2d(...)
    (3): ReLU(...)
    (4): MaxPool2d(...)
```

Encoder

[12]:
```
class VGG13Encoder(torch.nn.Module):
    def __init__(self, num_blocks, weights=VGG13_Weights.DEFAULT):
        """Build the first ``num_blocks`` VGG-13 feature stages as an encoder.

        Args:
            num_blocks (int): Number of encoder stages (each ends before its␣
↪pooling).
            weights (VGG13_Weights): Pretrained weights enum for ``torchvision.
↪models.vgg13``.

        Returns:
```

```
            None
        """
        super().__init__()
        self.num_blocks = num_blocks

        feature_extractor = vgg13(weights=weights).features

        self.blocks = torch.nn.ModuleList()
        for idx in range(self.num_blocks):
            start_idx = idx * 5
            end_idx = start_idx + 4
            layers = list(feature_extractor.children())[start_idx:end_idx]

            self.blocks.append(
                torch.nn.Sequential(*layers)
            )

    def forward(self, x):
        """Run the encoder and collect per-scale feature maps.

        Args:
            x (torch.Tensor): Input batch, shape ``[N, 3, H, W]``.

        Returns:
            list[torch.Tensor]: Feature map after each block (after pooling␣
↪except last).
        """
        activations = []
        for idx, block in enumerate(self.blocks):
            x = block(x)
            activations.append(x)

            if idx < self.num_blocks - 1:
                x = torch.nn.functional.max_pool2d(x, kernel_size=2, stride=2)

        return activations
```

[13]:
```
def run_vgg13_encoder_smoke_tests():
    """Parameter counts and forward norms for ``VGG13Encoder``.

    Args:
        None

    Returns:
        None
    """
```

```
    assert sum((param.numel() for param in VGG13Encoder(num_blocks=1).
↪parameters())) == 38720
    assert sum((param.numel() for param in VGG13Encoder(num_blocks=2).
↪parameters())) == 260160
    assert sum((param.numel() for param in VGG13Encoder(num_blocks=3).
↪parameters())) == 1145408
    x = torch.arange(1 * 3 * 320 * 240).reshape(1, 3, 320, 240) / (1 * 3 * 320␣
↪* 240)
    out = VGG13Encoder(num_blocks=3)(x)
    assert len(out) == 3
    npt.assert_almost_equal(torch.linalg.norm(out[0]).item(), 902.218,␣
↪decimal=3)
    npt.assert_almost_equal(torch.linalg.norm(out[1]).item(), 571.030,␣
↪decimal=3)
    npt.assert_almost_equal(torch.linalg.norm(out[2]).item(), 648.068,␣
↪decimal=3)


run_vgg13_encoder_smoke_tests()
```

Decoder

[14]:
```
class DecoderBlock(torch.nn.Module):
    def __init__(self, out_channels):
        """One U-Net decoder step: upsample, concat skip, two convs.

        Args:
            out_channels (int): Channel width after this block.

        Returns:
            None
        """
        super().__init__()

        self.upconv = torch.nn.Conv2d(
            in_channels=out_channels * 2, out_channels=out_channels,
            kernel_size=3, padding=1, dilation=1
        )
        self.conv1 = torch.nn.Conv2d(
            in_channels=out_channels * 2, out_channels=out_channels,
            kernel_size=3, padding=1, dilation=1
        )
        self.conv2 = torch.nn.Conv2d(
            in_channels=out_channels, out_channels=out_channels,
            kernel_size=3, padding=1, dilation=1
        )
        self.relu = torch.nn.ReLU()
```

```
    def forward(self, down, left):
        """Fuse upsampled ``down`` with encoder skip ``left``.

        Args:
            down (torch.Tensor): Deepest feature map being upsampled.
            left (torch.Tensor): Skip connection from encoder (same spatial␣
↪size as upsampled ``down``).

        Returns:
            torch.Tensor: Decoded features at this resolution.
        """
        x = F.interpolate(down, scale_factor=2, mode='nearest')
        x = self.upconv(x)

        x = torch.cat([left, x], dim=1)

        x = self.conv1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.relu(x)

        return x
```

[15]:
```
class Decoder(torch.nn.Module):
    def __init__(self, num_filters, num_blocks):
        """Stack of decoder blocks (used in some notebook experiments).

        Args:
            num_filters (int): Base filter count; widths scale by ``2**idx``.
            num_blocks (int): Number of ``DecoderBlock`` modules.

        Returns:
            None
        """
        super().__init__()

        self.blocks = torch.nn.ModuleList()
        for idx in range(num_blocks):
            self.blocks.insert(0, DecoderBlock(num_filters * 2 ** idx))

    def forward(self, acts):
        """Decode from deepest activation using stored encoder activations.

        Args:
            acts (list[torch.Tensor]): Encoder outputs, low-res last.
```

```
        Returns:
            torch.Tensor: Full-resolution feature map before final head.
        """
        up = acts[-1]
        for block, left in zip(self.blocks, acts[-2::-1]):
            up = block(up, left)
        return up
```

U-Net

[16]:
```
class UNet(torch.nn.Module):
    def __init__(self, num_classes=1, num_blocks=4):
        """U-Net with VGG13 encoder and concat-skip decoder.

        Args:
            num_classes (int): Number of output logits channels (1 for binary␣
↪mask).
            num_blocks (int): Encoder depth / number of VGG stages to use.

        Returns:
            None
        """
        super().__init__()
        self.channels = [64, 128, 256, 512, 512]

        self.encoder = VGG13Encoder(num_blocks=num_blocks)
        self.decoder = torch.nn.ModuleList()
        for i in range(num_blocks - 2, -1, -1):
            self.decoder.append(
                DecoderBlock(out_channels=self.channels[i])
            )

        self.final = torch.nn.Conv2d(
            in_channels=self.channels[0], out_channels=num_classes,␣
↪kernel_size=1
        )

    def forward(self, x):
        """Segment an image batch.

        Args:
            x (torch.Tensor): Input ``[N, 3, H, W]``.

        Returns:
            torch.Tensor: Logits ``[N, num_classes, H, W]``.
        """
        activations = self.encoder(x)
```

```
        x = activations[-1]

        for i, block in enumerate(self.decoder):
            left = activations[-(i + 2)]
            x = block(down=x, left=left)

        x = self.final(x)

        return x
```

[17]:
```
def run_unet_smoke_test():
    """Sanity-check ``UNet`` parameter count and output shape.

    Args:
        None

    Returns:
        UNet: The constructed model (for display / further use).
    """
    model = UNet(num_classes=1, num_blocks=3)
    x = torch.arange(1 * 3 * 320 * 240).reshape(1, 3, 320, 240) / (1 * 3 * 320␣
↪* 240)
    assert sum((param.numel() for param in model.parameters())) == 2067649
    assert list(model(x).shape) == [1, 1, 320, 240]
    return model


model = run_unet_smoke_test()
model
```

[17]:
```
UNet(
  (encoder): VGG13Encoder(
    (blocks): ModuleList(
      (0): Sequential(
        (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): ReLU(inplace=True)
        (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (3): ReLU(inplace=True)
      )
      (1): Sequential(
        (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): ReLU(inplace=True)
        (2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (3): ReLU(inplace=True)
      )
      (2): Sequential(
        (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
```

```
        (1): ReLU(inplace=True)
        (2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (3): ReLU(inplace=True)
      )
    )
  )
  (decoder): ModuleList(
    (0): DecoderBlock(
      (upconv): Conv2d(256, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (conv1): Conv2d(256, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (relu): ReLU()
    )
    (1): DecoderBlock(
      (upconv): Conv2d(128, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (conv1): Conv2d(128, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (relu): ReLU()
    )
  )
  (final): Conv2d(64, 1, kernel_size=(1, 1), stride=(1, 1))
)
```

Usually, the optimized functionality is difficult to interpret, and different loss functions can be used in different experiments. Therefore, it is necessary to measure and monitor an independent quality metric. It is suggested to use **IoU (Intersection over Union)** — one of the standard options for the segmentation task:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{\sum\limits_{ij} a_{ij} b_{ij}}{\sum\limits_{ij} a_{ij} + b_{ij} - a_{ij} b_{ij}}$$

Note that the denominator can be zero, for example, if the mask is zero. So projects can be implemented rather than studied, as otherwise metrics are calculated at all levels. There are built-in devices for charging using a mixer.

[18]:
```
class IoUScore(torch.nn.Module):
    def __init__(self, threshold, reduction=None):
        """Per-image IoU after thresholding (expects probabilities, not logits).

        Args:
            threshold (float): Binarization threshold on input values.
            reduction (str | None): ``'mean'``, ``'sum'``, or ``None``␣
↪(per-batch vector).
```

```
        Returns:
            None
        """
        super().__init__()

        self.threshold = threshold
        self.reduction = reduction

    @torch.no_grad()
    def forward(self, logits, true_labels):
        """Compute IoU between thresholded predictions and binarized targets.

        Args:
            logits (torch.Tensor): Probabilities (e.g. after sigmoid), shape␣
↪``[B, ...]``.
            true_labels (torch.Tensor): Ground-truth mask; values ``> 0.5``␣
↪treated as foreground.

        Returns:
            torch.Tensor: Scalar if ``reduction`` is set, else shape ``[B]``.
        """
        preds = (logits > self.threshold).view(logits.size(0), -1)
        true_labels = (true_labels > 0.5).view(true_labels.size(0), -1)

        intersection = (preds & true_labels).sum(dim=1).float()

        union = (preds | true_labels).sum(dim=1).float()

        eps = 1e-7
        score = (intersection + eps) / (union + eps)

        if self.reduction == 'sum':
            score = score.sum()
        elif self.reduction == 'mean':
            score = score.mean()

        return score
```

[19]:
```
def run_iou_score_unit_tests():
    """Assert ``IoUScore`` matches reference values for toy tensors.

    Args:
        None

    Returns:
        None
```

```
    """
    logits = torch.tensor(
        [
            [
                [0.3089, 0.4311, -0.9711],
                [0.9030, 1.0325, -0.7607],
                [0.9648, -0.5528, -1.1010],
            ],
            [
                [0.8906, 0.8099, 0.4458],
                [2.6215, -1.3198, 0.3142],
                [0.2262, -0.9175, -0.0174],
            ],
            [
                [0.8906, 0.8099, 0.4458],
                [2.6215, -1.3198, 0.3142],
                [0.2262, -0.9175, -0.0174],
            ],
        ]
    )
    true_labels = torch.tensor(
        [
            [[0.0, 1.0, 0.0], [1.0, 0.0, 0.0], [0.0, 0.0, 1.0]],
            [[1.0, 1.0, 1.0], [1.0, 0.0, 1.0], [1.0, 0.0, 0.0]],
            [[1.0, 1.0, 1.0], [0.0, 1.0, 1.0], [1.0, 1.0, 1.0]],
        ]
    )
    scores = IoUScore(0.0)(logits, true_labels)
    npt.assert_almost_equal(scores.numpy(), np.array([0.3333, 1.0000, 0.5556]),␣
↪decimal=4)
    score = IoUScore(0.0, reduction="sum")(logits, true_labels)
    npt.assert_almost_equal(score.item(), 1.8889, decimal=4)
    score = IoUScore(0.0, reduction="mean")(logits, true_labels)
    npt.assert_almost_equal(score.item(), 0.6296, decimal=4)


run_iou_score_unit_tests()
```

[20]:
```
def create_training_device_and_loaders(
    train_dataset,
    test_dataset,
    batch_size: int = 8,
):
    """Set ``device`` and default train/test loaders used by training helpers.

    Args:
        train_dataset (Dataset): Training set.
```

```
        test_dataset (Dataset): Test set.
        batch_size (int): Batch size.

    Returns:
        tuple: ``(device, train_loader, test_loader)``.
    """
    dev = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    tr_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,␣
 ↪drop_last=True)
    te_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
    return dev, tr_loader, te_loader


device, train_loader, test_loader =␣
 ↪create_training_device_and_loaders(train_dataset, test_dataset)


class CombinedLoss(torch.nn.Module):
    def __init__(self, bce_weight=1.0, dice_weight=1.0):
        """Weighted sum of BCE-with-logits and Dice losses.

        Args:
            bce_weight (float): Multiplier for BCE term.
            dice_weight (float): Multiplier for Dice term.

        Returns:
            None
        """
        super().__init__()
        self.bce = torch.nn.BCEWithLogitsLoss()
        self.dice = DiceLoss(with_logits=True, reduction="mean")
        self.bce_weight = bce_weight
        self.dice_weight = dice_weight

    def forward(self, logits, targets):
        """Combine weighted BCE-with-logits and Dice.

        Args:
            logits (torch.Tensor): Predicted logits, shape ``[N, 1, H, W]`` (or␣
 ↪compatible).
            targets (torch.Tensor): Target mask (float), same shape as logits.

        Returns:
            torch.Tensor: Scalar combined loss.
        """
        targets = targets.float()
        loss_bce = self.bce(logits, targets)
```

```
        loss_dice = self.dice(logits, targets)
        return self.bce_weight * loss_bce + self.dice_weight * loss_dice
```

[21]:
```
def train_and_evaluate(model, criterion, optimizer, epochs=5):
    """Train ``model`` on ``train_loader`` and evaluate on ``test_loader``.

    Uses module-level ``train_loader``, ``test_loader``, and ``device``.

    Args:
        model (torch.nn.Module): Segmentation model.
        criterion (torch.nn.Module): Loss taking ``(logits, masks)``.
        optimizer (torch.optim.Optimizer): Optimizer for ``model`` parameters.
        epochs (int): Number of training epochs.

    Returns:
        dict: History with keys ``train_loss``, ``test_loss``, ``train_iou``,␣
↪``test_iou``,
        ``train_acc``, ``test_acc`` (each a list of length ``epochs``).
    """
    history = {
        'train_loss': [], 'test_loss': [], 'train_iou': [], 'test_iou': [],
        'train_acc': [], 'test_acc': [],
    }
    iou_metric = IoUScore(threshold=0.5, reduction='mean').to(device)

    def pixel_accuracy(probs, masks):
        """Mean per-pixel agreement after thresholding.

        Args:
            probs (torch.Tensor): Predicted probabilities in ``[0, 1]``.
            masks (torch.Tensor): Target mask (float), foreground if ``> 0.5``.

        Returns:
            float: Accuracy in ``[0, 1]`` for this batch (scalar).
        """
        preds = (probs > 0.5).float()
        t = (masks > 0.5).float()
        return (preds == t).float().mean().item()

    for epoch in range(epochs):
        model.train()
        train_loss, train_iou, train_acc = 0, 0, 0

        for images, masks in tqdm(train_loader, desc=f'Epoch {epoch+1}/{epochs}␣
↪[Train]', leave=False):
            images, masks = images.to(device), masks.to(device)
```

```
            optimizer.zero_grad()
            logits = model(images)
            loss = criterion(logits, masks)
            loss.backward()
            optimizer.step()

            probs = torch.sigmoid(logits)
            train_loss += loss.item()
            train_iou += iou_metric(probs, masks).item()
            train_acc += pixel_accuracy(probs, masks)

        model.eval()
        test_loss, test_iou, test_acc = 0, 0, 0
        with torch.no_grad():
            for images, masks in tqdm(test_loader, desc=f'Epoch {epoch+1}/
↪{epochs} [Test]', leave=False):
                images, masks = images.to(device), masks.to(device)
                logits = model(images)
                loss = criterion(logits, masks)
                probs = torch.sigmoid(logits)

                test_loss += loss.item()
                test_iou += iou_metric(probs, masks).item()
                test_acc += pixel_accuracy(probs, masks)

        history['train_loss'].append(train_loss / len(train_loader))
        history['test_loss'].append(test_loss / len(test_loader))
        history['train_iou'].append(train_iou / len(train_loader))
        history['test_iou'].append(test_iou / len(test_loader))
        history['train_acc'].append(train_acc / len(train_loader))
        history['test_acc'].append(test_acc / len(test_loader))

        print(
            f"Epoch {epoch+1}: Train Loss={history['train_loss'][-1]:.4f}, Test␣
↪Loss={history['test_loss'][-1]:.4f} | "
            f"Test IoU={history['test_iou'][-1]:.4f}, Test Pixel␣
↪Acc={history['test_acc'][-1]:.4f}"
        )

    return history
```

[ ]:
```
def run_unet_loss_experiments(epochs: int = 5):
    """Train ``UNet`` under several BCE/Dice weightings; pick best by final␣
↪test IoU.

    Args:
        epochs (int): Epochs per experiment.
```

```
    Returns:
        tuple: ``(best_model, best_iou, all_histories, all_model_weights,␣
↪best_model_weights)``.
    """
    print(f"Using device: {device}")
    experiments = {
        "1. Only BCE": (1.0, 0.0),
        "2. Only Dice": (0.0, 1.0),
        "3. BCE(0.5) + Dice(0.5)": (0.5, 0.5),
        "4. BCE(0.2) + Dice(0.8)": (0.2, 0.8),
        "5. BCE(0.8) + Dice(0.2)": (0.8, 0.2),
    }
    all_histories = {}
    all_model_weights = {}
    best_model_weights = None
    best_iou = 0.0
    for exp_name, (w_bce, w_dice) in experiments.items():
        print(f"\n{'='*40}\nStarting experiment: {exp_name}\n{'='*40}")
        model = UNet(num_classes=1, num_blocks=4).to(device)
        criterion = CombinedLoss(bce_weight=w_bce, dice_weight=w_dice).
↪to(device)
        optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
        hist = train_and_evaluate(model, criterion, optimizer, epochs=epochs)
        all_histories[exp_name] = hist
        all_model_weights[exp_name] = model.state_dict()
        final_test_iou = hist["test_iou"][-1]
        if final_test_iou > best_iou:
            best_iou = final_test_iou
            best_model_weights = model.state_dict()
    best_model = UNet(num_classes=1, num_blocks=4).to(device)
    best_model.load_state_dict(best_model_weights)
    print(f"\nThe best strategy achieved a Test IoU = {best_iou:.4f}")
    return best_model, best_iou, all_histories, all_model_weights,␣
↪best_model_weights

best_model, best_iou, all_histories, all_model_weights, best_model_weights =␣
↪run_unet_loss_experiments()
```

```
Using device: cuda

========================================
Starting experiment: 1. Only BCE
========================================


Epoch 1: Train Loss=0.5745, Test Loss=0.5225 | Test IoU=0.6634, Test Pixel
```

```
Acc=0.7513


Epoch 2: Train Loss=0.5530, Test Loss=0.5322 | Test IoU=0.6380, Test Pixel
Acc=0.7368


Epoch 3: Train Loss=0.5013, Test Loss=0.5072 | Test IoU=0.6812, Test Pixel
Acc=0.7562


Epoch 4: Train Loss=0.4958, Test Loss=0.5159 | Test IoU=0.6681, Test Pixel
Acc=0.7568


Epoch 5: Train Loss=0.4863, Test Loss=0.5206 | Test IoU=0.6939, Test Pixel
Acc=0.7674

========================================
Starting experiment: 2. Only Dice
========================================


Epoch 1: Train Loss=0.2814, Test Loss=0.2595 | Test IoU=0.6045, Test Pixel
Acc=0.6045


Epoch 2: Train Loss=0.2803, Test Loss=0.2595 | Test IoU=0.6045, Test Pixel
Acc=0.6045


Epoch 3: Train Loss=0.2802, Test Loss=0.2595 | Test IoU=0.6045, Test Pixel
Acc=0.6045


Epoch 4: Train Loss=0.2798, Test Loss=0.2595 | Test IoU=0.6045, Test Pixel
Acc=0.6045


Epoch 5: Train Loss=0.2803, Test Loss=0.2595 | Test IoU=0.6045, Test Pixel
Acc=0.6045

========================================
Starting experiment: 3. BCE(0.5) + Dice(0.5)
========================================
```

```
Epoch 1: Train Loss=0.6574, Test Loss=0.3914 | Test IoU=0.6867, Test Pixel
Acc=0.7698


Epoch 2: Train Loss=0.3653, Test Loss=0.3575 | Test IoU=0.7019, Test Pixel
Acc=0.7879


Epoch 3: Train Loss=0.3383, Test Loss=0.3263 | Test IoU=0.7264, Test Pixel
Acc=0.8080


Epoch 4: Train Loss=0.3188, Test Loss=0.3104 | Test IoU=0.7452, Test Pixel
Acc=0.8159


Epoch 5: Train Loss=0.3073, Test Loss=0.3158 | Test IoU=0.7316, Test Pixel
Acc=0.8191

========================================
Starting experiment: 4. BCE(0.2) + Dice(0.8)
========================================


Epoch 1: Train Loss=0.4102, Test Loss=0.2956 | Test IoU=0.6970, Test Pixel
Acc=0.7674


Epoch 2: Train Loss=0.2786, Test Loss=0.2678 | Test IoU=0.7173, Test Pixel
Acc=0.7948


Epoch 3: Train Loss=0.2613, Test Loss=0.2600 | Test IoU=0.7317, Test Pixel
Acc=0.8009


Epoch 4: Train Loss=0.2489, Test Loss=0.2407 | Test IoU=0.7394, Test Pixel
Acc=0.8176


Epoch 5: Train Loss=0.2436, Test Loss=0.2680 | Test IoU=0.7193, Test Pixel
Acc=0.8051

========================================
Starting experiment: 5. BCE(0.8) + Dice(0.2)
========================================
```

```
Epoch 1: Train Loss=0.6215, Test Loss=0.4889 | Test IoU=0.6499, Test Pixel
Acc=0.7441


Epoch 2: Train Loss=0.4919, Test Loss=0.4694 | Test IoU=0.6592, Test Pixel
Acc=0.7468


Epoch 3: Train Loss=0.4561, Test Loss=0.4338 | Test IoU=0.6948, Test Pixel
Acc=0.7752


Epoch 4: Train Loss=0.3751, Test Loss=0.3544 | Test IoU=0.7583, Test Pixel
Acc=0.8222


Epoch 5: Train Loss=0.3158, Test Loss=0.2776 | Test IoU=0.8049, Test Pixel
Acc=0.8722

The best strategy achieved a Test IoU = 0.8049
```

1.1 Save model

Persist Khoroshilova model weights and training histories under `checkpoints/`.

[23]:
```
def save_unet_experiment_artifacts(
    checkpoint_dir=None,
    all_model_weights=None,
    best_model_weights=None,
    all_histories=None,
):
    """Save per-experiment weights, optional best-only fallback, and history␣
 ↪JSON.

    Args:
        checkpoint_dir (pathlib.Path | None): Output directory; default ``cwd /␣
 ↪"checkpoints"``.
        all_model_weights (dict | None): Map experiment name -> state dict.
        best_model_weights (dict | None): Best single state dict.
        all_histories (dict | None): Training histories to JSON.

    Returns:
        pathlib.Path: ``checkpoint_dir`` used.
    """
    if checkpoint_dir is None:
        checkpoint_dir = Path.cwd() / "checkpoints"
    checkpoint_dir.mkdir(parents=True, exist_ok=True)
    if all_model_weights:
```

```
        for name, weights in all_model_weights.items():
            fname = f"khoroshilova_{name.replace(' ', '_')}.pt"
            torch.save(weights, checkpoint_dir / fname)
            print(f"Saved: {fname}")
    elif best_model_weights is not None:
        torch.save(best_model_weights, checkpoint_dir / "khoroshilova_best_unet.
↪pt")
        print("Saved: khoroshilova_best_unet.pt")
    else:
        print("No model weights found; run the experiments cell first.")
    if all_histories:
        with open(checkpoint_dir / "khoroshilova_all_histories.json", "w") as f:
            json.dump(all_histories, f, indent=2)
        print("Saved: khoroshilova_all_histories.json")
    return checkpoint_dir


CHECKPOINT_DIR = save_unet_experiment_artifacts(
    all_model_weights=all_model_weights,
    best_model_weights=best_model_weights,
    all_histories=all_histories,
)
```

```
Saved: khoroshilova_1._Only_BCE.pt
Saved: khoroshilova_2._Only_Dice.pt
Saved: khoroshilova_3._BCE(0.5)_+_Dice(0.5).pt
Saved: khoroshilova_4._BCE(0.2)_+_Dice(0.8).pt
Saved: khoroshilova_5._BCE(0.8)_+_Dice(0.2).pt
Saved: khoroshilova_all_histories.json
```

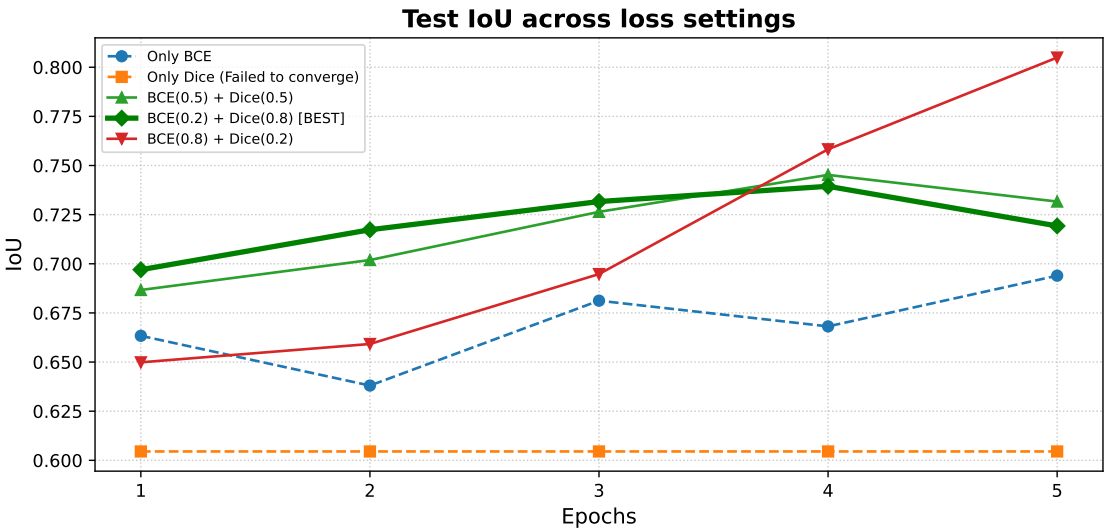
IoU (Intersection over Union) compares the predicted binary mask A and the ground-truth mask B: $\text{IoU} = |A \cap B|/|A \cup B|$. It measures overlap (1 = perfect). **Pixel accuracy** is the fraction of pixels where the predicted class matches the label (0.5 threshold on probabilities). With lots of background, pixel accuracy can look high even when the foreground is wrong, so IoU is often the more informative metric for segmentation.

[24]:
```
def plot_unet_loss_experiment_curves(all_histories=None):
    """Plot IoU / accuracy / loss curves for loss ablation (live or fallback␣
↪data).

    Args:
        all_histories (dict | None): If None, uses global ``all_histories`` if␣
↪present.

    Returns:
        None
    """
    epochs = [1, 2, 3, 4, 5]
```

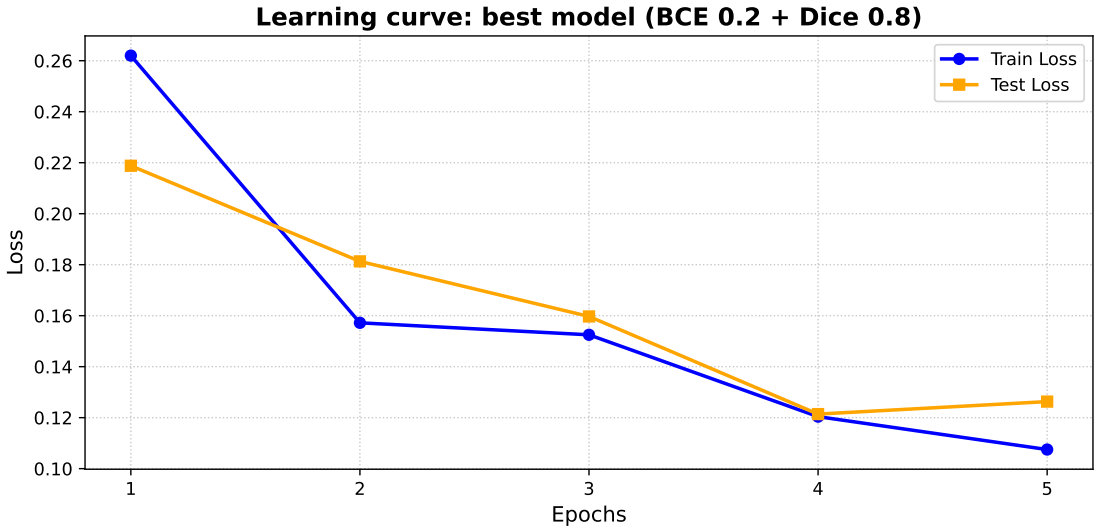
```
    exp_names = [
        "1. Only BCE",
        "2. Only Dice",
        "3. BCE(0.5) + Dice(0.5)",
        "4. BCE(0.2) + Dice(0.8)",
        "5. BCE(0.8) + Dice(0.2)",
    ]
    iou_exp1 = [0.7233, 0.7161, 0.8033, 0.8278, 0.8189]
    iou_exp2 = [0.6045, 0.6045, 0.6045, 0.6045, 0.6045]
    iou_exp3 = [0.6546, 0.7572, 0.7770, 0.7926, 0.8392]
    iou_exp4 = [0.7966, 0.8284, 0.8441, 0.8677, 0.8606]
    iou_exp5 = [0.6931, 0.7075, 0.6895, 0.7125, 0.7346]
    train_loss_best = [0.2620, 0.1572, 0.1525, 0.1204, 0.1075]
    test_loss_best = [0.2188, 0.1813, 0.1597, 0.1214, 0.1263]
    _hist = all_histories if all_histories is not None else globals().
↪get("all_histories")

    def _history_rows(metric_key, fallback_rows):
        """Build per-experiment metric rows from ``all_histories`` or static␣
↪fallbacks.

        Args:
            metric_key (str): Key inside each experiment dict (e.g.␣
↪``"test_iou"``).
            fallback_rows (list | None): Default rows if histories lack keys.

        Returns:
            list | None: List of epoch series, or None if unavailable.
        """
        if _hist and all(name in _hist and metric_key in _hist[name] for name␣
↪in exp_names):
            return [_hist[name][metric_key] for name in exp_names]
        return fallback_rows

    iou_rows = _history_rows("test_iou", [iou_exp1, iou_exp2, iou_exp3,␣
↪iou_exp4, iou_exp5])
    acc_rows = _history_rows("test_acc", None)
    plot_styles = [
        ("Only BCE", "o", "--", None, None),
        ("Only Dice (Failed to converge)", "s", "--", None, None),
        ("BCE(0.5) + Dice(0.5)", "^", "-", None, None),
        ("BCE(0.2) + Dice(0.8) [BEST]", "D", "-", 3, "green"),
        ("BCE(0.8) + Dice(0.2)", "v", "-", None, None),
    ]
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(9, 13))
    for row, (lab, m, ls, lw, c) in zip(iou_rows, plot_styles):
        kw = {"marker": m, "linestyle": ls, "label": lab}
```

```
        if lw is not None:
            kw["linewidth"] = lw
        if c is not None:
            kw["color"] = c
        ax1.plot(epochs, row, **kw)
    ax1.set_title("Test IoU across loss settings", fontsize=14,␣
↪fontweight="bold")
    ax1.set_xlabel("Epochs", fontsize=12)
    ax1.set_ylabel("IoU", fontsize=12)
    ax1.set_xticks(epochs)
    ax1.grid(True, linestyle=":", alpha=0.7)
    ax1.legend(loc="best", fontsize=8)
    if acc_rows is not None:
        for row, (lab, m, ls, lw, c) in zip(acc_rows, plot_styles):
            kw = {"marker": m, "linestyle": ls, "label": lab}
            if lw is not None:
                kw["linewidth"] = lw
            if c is not None:
                kw["color"] = c
            ax2.plot(epochs, row, **kw)
        ax2.set_title("Test pixel accuracy across loss settings", fontsize=14,␣
↪fontweight="bold")
        ax2.set_xlabel("Epochs", fontsize=12)
        ax2.set_ylabel("Pixel accuracy", fontsize=12)
        ax2.set_xticks(epochs)
        ax2.set_ylim(0, 1.02)
        ax2.grid(True, linestyle=":", alpha=0.7)
        ax2.legend(loc="best", fontsize=8)
    else:
        ax2.text(
            0.5,
            0.5,
            "Pixel accuracy: re-run the loss experiments so\nall_histories␣
↪contains test_acc (see train_and_evaluate).",
            ha="center",
            va="center",
            transform=ax2.transAxes,
            fontsize=11,
        )
        ax2.set_axis_off()
    ax3.plot(epochs, train_loss_best, marker="o", color="blue", linewidth=2,␣
↪label="Train Loss")
    ax3.plot(epochs, test_loss_best, marker="s", color="orange", linewidth=2,␣
↪label="Test Loss")
    ax3.set_title("Learning curve: best model (BCE 0.2 + Dice 0.8)",␣
↪fontsize=14, fontweight="bold")
```

```
    ax3.set_xlabel("Epochs", fontsize=12)
    ax3.set_ylabel("Loss", fontsize=12)
    ax3.set_xticks(epochs)
    ax3.grid(True, linestyle=":", alpha=0.7)
    ax3.legend()
    plt.tight_layout()
    plt.show()


plot_unet_loss_experiment_curves(all_histories=all_histories)
```


Test IoU across loss settings
Only BCE
Only Dice (Failed to converge)
BCE(0.5) + Dice(0.5)
BCE(0.2) + Dice(0.8) [BEST]
BCE(0.8) + Dice(0.2)
IoU
0.800
0.775
0.750
0.725
0.700
0.675
0.650
0.625
0.600
1
2
3
4
5
Epochs

Test pixel accuracy across loss settings
Pixel accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Only BCE
Only Dice (Failed to converge)
BCE(0.5) + Dice(0.5)
BCE(0.2) + Dice(0.8) [BEST]
BCE(0.8) + Dice(0.2)
1
2
3
4
5
Epochs


Learning curve: best model (BCE 0.2 + Dice 0.8)
Train Loss
Test Loss
Loss
0.26
0.24
0.22
0.20
0.18
0.16
0.14
0.12
0.10
1
2
3
4
5
Epochs

[23]:
```
class LinkNetDecoderBlock(torch.nn.Module):
    def __init__(self, in_channels, out_channels):
        """Decoder block with additive skip connection (LinkNet-style).

        Args:
            in_channels (int): Channels on the upsampled deep feature map.
            out_channels (int): Output channels for this level.

        Returns:
            None
        """
        super().__init__()
        self.upconv = torch.nn.Conv2d(
            in_channels=in_channels, out_channels=out_channels,
            kernel_size=3, padding=1
        )

        self.conv1 = torch.nn.Conv2d(
            in_channels=out_channels, out_channels=out_channels,
            kernel_size=3, padding=1
        )
        self.conv2 = torch.nn.Conv2d(
            in_channels=out_channels, out_channels=out_channels,
            kernel_size=3, padding=1
        )
        self.relu = torch.nn.ReLU()

    def forward(self, down, left):
        """Upsample ``down``, add skip ``left``, then convolve.

        Args:
            down (torch.Tensor): Deep features to upsample.
            left (torch.Tensor): Encoder skip (same spatial size as upsampled␣
↪``down``).

        Returns:
            torch.Tensor: Features for this decoder level.
        """
        x = F.interpolate(down, scale_factor=2, mode='nearest')
        x = self.upconv(x)
        x = self.relu(x)

        x = x + left

        x = self.conv1(x)
        x = self.relu(x)
        x = self.conv2(x)
```

```python
        x = self.relu(x)

        return x

class LinkNet(torch.nn.Module):
    def __init__(self, num_classes=1, num_blocks=4):
        """LinkNet decoder + VGG13 encoder (same layout as ``UNet`` but␣
 ↪additive skips).

        Args:
            num_classes (int): Output channels for the 1x1 head.
            num_blocks (int): Encoder depth (VGG stages).

        Returns:
            None
        """
        super().__init__()
        self.channels = [64, 128, 256, 512, 512]

        self.encoder = VGG13Encoder(num_blocks=num_blocks)

        self.decoder = torch.nn.ModuleList()
        for i in range(num_blocks - 2, -1, -1):
            self.decoder.append(
                LinkNetDecoderBlock(
                    in_channels=self.channels[i + 1],
                    out_channels=self.channels[i]
                )
            )

        self.final = torch.nn.Conv2d(
            in_channels=self.channels[0], out_channels=num_classes,␣
 ↪kernel_size=1
        )

    def forward(self, x):
        """Encode with VGG13, decode with additive skips, then 1x1 head.

        Args:
            x (torch.Tensor): Input ``[N, 3, H, W]``.

        Returns:
            torch.Tensor: Logits ``[N, num_classes, H, W]``.
        """
        activations = self.encoder(x)
        x = activations[-1]
```

```
        for i, block in enumerate(self.decoder):
            left = activations[-(i + 2)]
            x = block(down=x, left=left)

        x = self.final(x)
        return x
```

[26]:
```
def train_linknet_and_report():
    """Compare parameter counts and train ``LinkNet`` with fixed combined loss.

    Args:
        None

    Returns:
        tuple: ``(linknet_model, linknet_history)``.
    """
    dev = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f'Using {dev}.\n')
    unet_params = sum(p.numel() for p in UNet().parameters() if p.requires_grad)
    linknet_params = sum(p.numel() for p in LinkNet().parameters() if p.
 ↪requires_grad)
    print(f"Parameters U-Net:   {unet_params:,}")
    print(f"Parameters LinkNet: {linknet_params:,}")
    print(f"LinkNet is lighter by {unet_params - linknet_params:,} Parameters!
 ↪\n")
    print("Starting learning LinkNet...")
    linknet_model = LinkNet(num_classes=1, num_blocks=4).to(dev)
    criterion = CombinedLoss(bce_weight=0.5, dice_weight=0.5).to(dev)
    optimizer = torch.optim.Adam(linknet_model.parameters(), lr=1e-3)
    linknet_history = train_and_evaluate(linknet_model, criterion, optimizer,␣
 ↪epochs=5)
    return linknet_model, linknet_history

linknet_model, linknet_history = train_linknet_and_report()
```

```
Using cuda.

Parameters U-Net:   8,557,505
Parameters LinkNet: 7,783,361
LinkNet is lighter by 774,144 Parameters!

Starting learning LinkNet…


Epoch 1: Train Loss=0.4288, Test Loss=0.3806 | Test IoU=0.7140, Test Pixel
Acc=0.7770
```

```
Epoch 2: Train Loss=0.2816, Test Loss=0.2342 | Test IoU=0.8098, Test Pixel
Acc=0.8718


Epoch 3: Train Loss=0.2136, Test Loss=0.1880 | Test IoU=0.8471, Test Pixel
Acc=0.9033


Epoch 4: Train Loss=0.1764, Test Loss=0.1753 | Test IoU=0.8560, Test Pixel
Acc=0.9106


Epoch 5: Train Loss=0.1657, Test Loss=0.1701 | Test IoU=0.8608, Test Pixel
Acc=0.9107
```

[27]:
```
def save_linknet_artifacts(linknet_model, linknet_history, checkpoint_dir=None):
    """Persist LinkNet weights and history JSON.

    Args:
        linknet_model (torch.nn.Module | None): Model to save, or None to skip.
        linknet_history (dict | None): History dict, or None to skip JSON.
        checkpoint_dir (pathlib.Path | None): Output directory.

    Returns:
        pathlib.Path: Directory used.
    """
    if checkpoint_dir is None:
        checkpoint_dir = Path.cwd() / "checkpoints"
    checkpoint_dir.mkdir(parents=True, exist_ok=True)
    if linknet_model is not None:
        torch.save(linknet_model.state_dict(), checkpoint_dir /␣
↪"khoroshilova_linknet.pt")
        print("Saved: khoroshilova_linknet.pt")
    else:
        print("No linknet_model passed; run the LinkNet training cell first.")
    if linknet_history is not None:
        with open(checkpoint_dir / "khoroshilova_linknet_history.json", "w") as␣
↪f:
            json.dump(linknet_history, f, indent=2)
        print("Saved: khoroshilova_linknet_history.json")
    return checkpoint_dir


CHECKPOINT_DIR = save_linknet_artifacts(linknet_model, linknet_history)
```

```
Saved: khoroshilova_linknet.pt
```

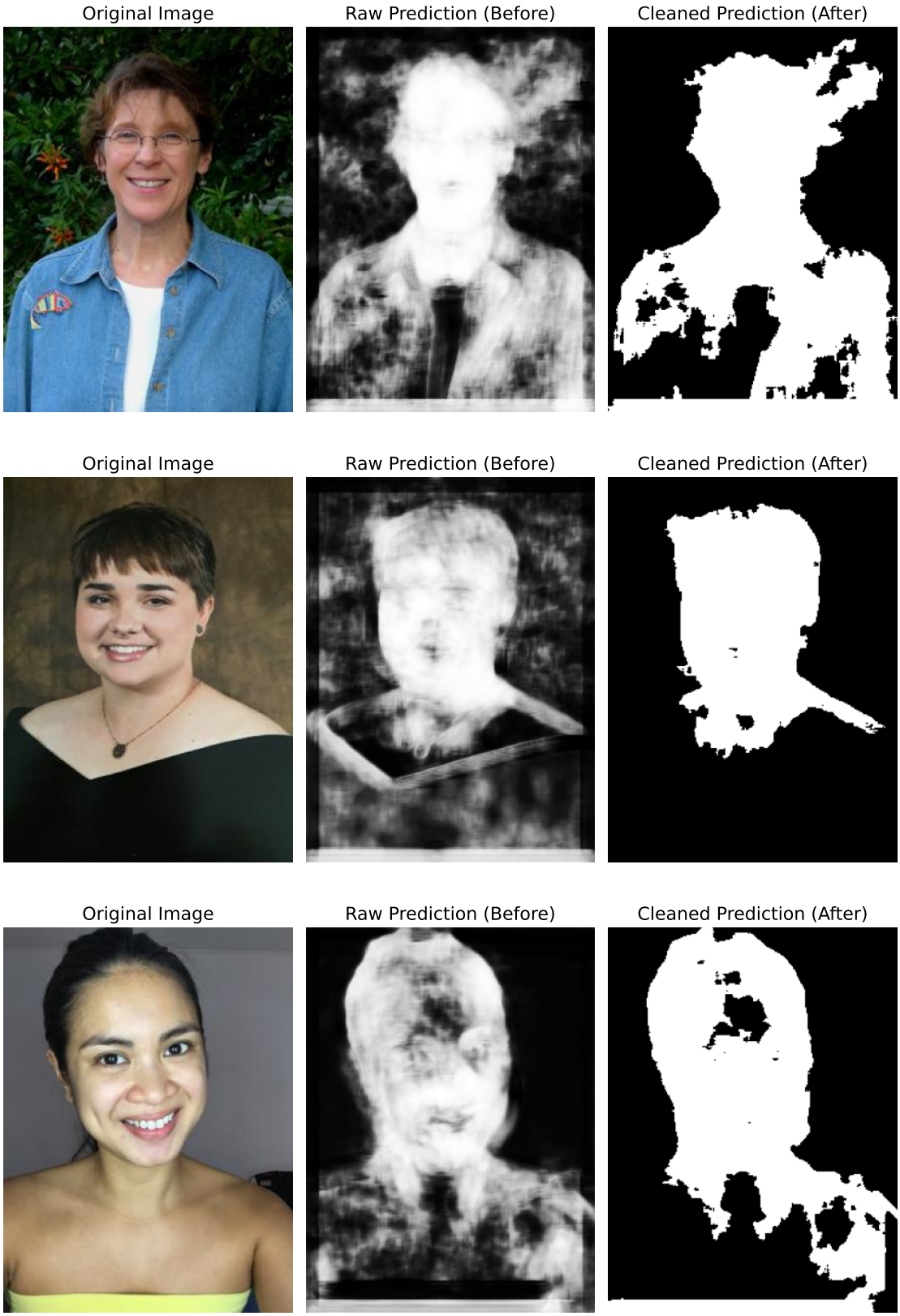
```
Saved: khoroshilova_linknet_history.json
```

[28]:
```python
def plot_linknet_predictions_grid(
    model,
    data_loader,
    device,
    num_images: int = 3,
):
    """Show original, ground truth, and binarized LinkNet predictions.

    Args:
        model (torch.nn.Module): Trained ``LinkNet``.
        data_loader (DataLoader): Batched test data.
        device (torch.device): Device for inference.
        num_images (int): Number of rows to plot.

    Returns:
        None
    """
    model.eval()
    images, true_masks = next(iter(data_loader))
    images = images.to(device)
    with torch.no_grad():
        logits = model(images)
        preds = (torch.sigmoid(logits) > 0.5).float()
    images_np = images.cpu().permute(0, 2, 3, 1).numpy()
    true_masks_np = true_masks.cpu().squeeze(1).numpy()
    preds_np = preds.cpu().squeeze(1).numpy()
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    fig, axes = plt.subplots(num_images, 3, figsize=(10, 4 * num_images))
    for i in range(num_images):
        img = np.clip(images_np[i] * std + mean, 0, 1)
        axes[i, 0].imshow(img)
        axes[i, 0].set_title("Original Photo", fontsize=14)
        axes[i, 0].axis("off")
        axes[i, 1].imshow(true_masks_np[i], cmap="gray")
        axes[i, 1].set_title("True Mask", fontsize=14)
        axes[i, 1].axis("off")
        axes[i, 2].imshow(preds_np[i], cmap="gray")
        axes[i, 2].set_title("LinkNet Prediction", fontsize=14)
        axes[i, 2].axis("off")
    plt.tight_layout()
    plt.show()


plot_linknet_predictions_grid(linknet_model, test_loader, device, num_images=3)
```

[30]:
```
def postprocess_mask(mask_np):
    """Keep only the largest connected component above 0.5 (noise removal).

    Args:
```

```
        mask_np (numpy.ndarray): 2D float array of probabilities or logits-like␣
↪values.

    Returns:
        numpy.ndarray: Float32 binary mask ``{0, 1}`` of the largest region, or␣
↪``mask_np`` if empty.
    """
    labels = measure.label(mask_np > 0.5)
    if labels.max() == 0:
        return mask_np
    region_sizes = np.bincount(labels.flat)
    largest_region_id = np.argmax(region_sizes[1:]) + 1
    cleaned_mask = (labels == largest_region_id).astype(np.float32)
    return cleaned_mask


def plot_linknet_postprocessed_grid(model, data_loader, device, num_images: int␣
↪= 3):
    """Compare raw sigmoid predictions vs largest-component post-processing.

    Args:
        model (torch.nn.Module): Trained ``LinkNet``.
        data_loader (DataLoader): Test batches.
        device (torch.device): Inference device.
        num_images (int): Rows to display.

    Returns:
        None
    """
    model.eval()
    images, true_masks = next(iter(data_loader))
    images = images.to(device)
    with torch.no_grad():
        logits = model(images)
        preds = torch.sigmoid(logits)
    images_np = images.cpu().permute(0, 2, 3, 1).numpy()
    preds_np = preds.cpu().squeeze(1).numpy()
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    fig, axes = plt.subplots(num_images, 3, figsize=(10, 5 * num_images))
    for i in range(num_images):
        img = np.clip(images_np[i] * std + mean, 0, 1)
        raw_prediction = preds_np[i]
        cleaned_prediction = postprocess_mask(raw_prediction)
        axes[i, 0].imshow(img)
        axes[i, 0].set_title("Original Image", fontsize=14)
        axes[i, 0].axis("off")
```

```
        axes[i, 1].imshow(raw_prediction, cmap="gray")
        axes[i, 1].set_title("Raw Prediction (Before)", fontsize=14)
        axes[i, 1].axis("off")
        axes[i, 2].imshow(cleaned_prediction, cmap="gray")
        axes[i, 2].set_title("Cleaned Prediction (After)", fontsize=14)
        axes[i, 2].axis("off")
    plt.tight_layout()
    plt.show()


plot_linknet_postprocessed_grid(linknet_model, test_loader, device,␣
 ↪num_images=3)
```


Original Image
Raw Prediction (Before)
Cleaned Prediction (After)
Original Image
Raw Prediction (Before)
Cleaned Prediction (After)
Original Image
Raw Prediction (Before)
Cleaned Prediction (After)

2 Safwan Hossain

2.1 Encoder Fine-Tuning Strategy Study

The previous section trained both **U-Net** and **LinkNet** end-to-end, meaning every layer (including the pretrained VGG13 encoder) was updated by the optimizer from the first step.

This section investigates **how many encoder blocks should be frozen** during training. Freezing early layers preserves low-level ImageNet features (edges, textures) and reduces the number of trainable parameters, while unfreezing deeper layers lets the model adapt high-level representations to the segmentation task.

We also swap in a **regularized decoder** (BatchNorm + Dropout) to see whether it improves generalization alongside the freeze strategy.

Experiment	Frozen blocks	Decoder
Full fine-tune	0	Regularized
Freeze 1 block	1	Regularized
Freeze 2 blocks	2	Regularized
Freeze 3 blocks	3	Regularized

2.1.1 Utility: freezing encoder blocks

`freeze_encoder_blocks(model, num_blocks_to_freeze)` disables gradient computation for the first N encoder blocks, keeping their pretrained ImageNet weights fixed during training.

[31]:
```
def freeze_encoder_blocks(model, num_blocks_to_freeze):
    """Freeze the first ``num_blocks_to_freeze`` VGG13 encoder blocks␣
↪(``requires_grad=False``).

    Args:
        model (torch.nn.Module): Model with ``model.encoder.blocks`` (e.g.␣
↪``UNet``, ``LinkNet``).
        num_blocks_to_freeze (int): Number of initial encoder blocks to freeze;␣
↪``0`` freezes none.

    Returns:
        None: Prints trainable vs total parameter counts as a side effect.
    """
    for idx, block in enumerate(model.encoder.blocks):
        requires_grad = idx >= num_blocks_to_freeze
        for param in block.parameters():
            param.requires_grad = requires_grad

    frozen = sum(1 for b in model.encoder.blocks
                 if not next(b.parameters()).requires_grad)
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    print(
```

```
        f"Encoder blocks frozen: {frozen}/{len(model.encoder.blocks)} | "
        f"Trainable params: {trainable:,} / {total:,}"
    )
```

2.1.2 Regularized decoder: BatchNorm + Dropout

We add **Batch Normalization** after each convolution (stabilizes training, especially when parts of the encoder are frozen) and **Dropout** (reduces overfitting when few parameters are being trained).

The encoder stays the same pretrained `VGG13Encoder`; only the decoder blocks change.

[33]:
```
class RegularizedDecoderBlock(torch.nn.Module):
    """Decoder block with BatchNorm after each conv and optional Dropout."""

    def __init__(self, out_channels, dropout_p=0.2):
        """Build conv-BN-ReLU stack with optional spatial dropout.

        Args:
            out_channels (int): Channel width for convolutions in this block.
            dropout_p (float): Dropout probability for ``Dropout2d``.

        Returns:
            None
        """
        super().__init__()
        self.upconv = torch.nn.Conv2d(
            out_channels * 2, out_channels, kernel_size=3, padding=1,
        )
        self.bn0 = torch.nn.BatchNorm2d(out_channels)
        self.conv1 = torch.nn.Conv2d(
            out_channels * 2, out_channels, kernel_size=3, padding=1,
        )
        self.bn1 = torch.nn.BatchNorm2d(out_channels)
        self.conv2 = torch.nn.Conv2d(
            out_channels, out_channels, kernel_size=3, padding=1,
        )
        self.bn2 = torch.nn.BatchNorm2d(out_channels)
        self.relu = torch.nn.ReLU()
        self.dropout = torch.nn.Dropout2d(p=dropout_p)

    def forward(self, down, left):
        """U-Net-style upsample + concat skip with BN and dropout.

        Args:
            down (torch.Tensor): Deep features to upsample.
            left (torch.Tensor): Encoder skip tensor.

        Returns:
```

```
            torch.Tensor: Decoded features at this resolution.
        """
        x = F.interpolate(down, scale_factor=2, mode="nearest")
        x = self.relu(self.bn0(self.upconv(x)))
        x = torch.cat([left, x], dim=1)
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.dropout(x)
        x = self.relu(self.bn2(self.conv2(x)))
        return x


class UNetRegularized(torch.nn.Module):
    """U-Net with the same VGG13 encoder but a regularized decoder."""

    def __init__(self, num_classes=1, num_blocks=4, dropout_p=0.2):
        """Same encoder as ``UNet`` with batch-norm and dropout in the decoder.

        Args:
            num_classes (int): Channels for the final 1x1 conv.
            num_blocks (int): Encoder depth (VGG stages).
            dropout_p (float): Dropout rate in each ``RegularizedDecoderBlock``.

        Returns:
            None
        """
        super().__init__()
        self.channels = [64, 128, 256, 512, 512]
        self.encoder = VGG13Encoder(num_blocks=num_blocks)
        self.decoder = torch.nn.ModuleList()
        for i in range(num_blocks - 2, -1, -1):
            self.decoder.append(
                RegularizedDecoderBlock(
                    out_channels=self.channels[i], dropout_p=dropout_p,
                )
            )
        self.final = torch.nn.Conv2d(self.channels[0], num_classes,␣
↪kernel_size=1)

    def forward(self, x):
        """Run encoder, regularized decoder, and classification head.

        Args:
            x (torch.Tensor): Input ``[N, 3, H, W]``.

        Returns:
            torch.Tensor: Logits ``[N, num_classes, H, W]``.
        """
```

```
        activations = self.encoder(x)
        x = activations[-1]
        for i, block in enumerate(self.decoder):
            left = activations[-(i + 2)]
            x = block(down=x, left=left)
        return self.final(x)


def verify_unet_regularized_shape():
    """Assert output shape and print parameter count for ``UNetRegularized``.

    Args:
        None

    Returns:
        None
    """
    _test_model = UNetRegularized(num_classes=1, num_blocks=4)
    _x = torch.randn(1, 3, 320, 240)
    assert list(_test_model(_x).shape) == [1, 1, 320, 240]
    print(f"UNetRegularized params: {sum(p.numel() for p in _test_model.
 ↪parameters()):,}")
    del _test_model, _x


verify_unet_regularized_shape()
```

```
UNetRegularized params: 8,560,193
```

2.1.3 Fine-tuning experiments

We train `UNetRegularized` under **4 freeze strategies**, all using the best loss from Khoroshilova's sweep (**BCE 0.2 + Dice 0.8**) and `Adam(lr=1e-3)` for 5 epochs.

Label	Blocks frozen
Full fine-tune	0 (baseline — all encoder layers trainable)
Freeze 1 block	1 (first 64-channel block frozen)
Freeze 2 blocks	2 (64 + 128 blocks frozen)
Freeze all encoder	all encoder blocks frozen, only decoder + head train

[34]:
```
def run_freeze_strategy_experiments(num_blocks: int = 4, epochs: int = 5):
    """Train ``UNetRegularized`` with several encoder freeze depths; track GPU␣
 ↪memory.

    Args:
        num_blocks (int): Encoder depth passed to the model.
        epochs (int): Training epochs per strategy.
```

```
    Returns:
        tuple: ``(freeze_histories, freeze_best_weights, freeze_best_iou,␣
↪freeze_memory, freeze_best_strategy)``.
    """
    freeze_strategies = {
        "Full fine-tune (freeze 0)": 0,
        "Freeze 1 block": 1,
        "Freeze 2 blocks": 2,
        f"Freeze all encoder ({num_blocks} blocks)": num_blocks,
    }
    freeze_histories = {}
    freeze_best_weights = None
    freeze_best_iou = 0.0
    freeze_memory = {}
    freeze_best_strategy = None
    for strategy_name, n_freeze in freeze_strategies.items():
        print(f"\n{'='*50}")
        print(f"Strategy: {strategy_name}")
        print(f"{'='*50}")
        model = UNetRegularized(num_classes=1, num_blocks=num_blocks).to(device)
        freeze_encoder_blocks(model, n_freeze)
        trainable_params = [p for p in model.parameters() if p.requires_grad]
        criterion = CombinedLoss(bce_weight=0.2, dice_weight=0.8).to(device)
        optimizer = torch.optim.Adam(trainable_params, lr=1e-3)
        if torch.cuda.is_available():
            torch.cuda.reset_peak_memory_stats(device)
        hist = train_and_evaluate(model, criterion, optimizer, epochs=epochs)
        freeze_histories[strategy_name] = hist
        if torch.cuda.is_available():
            peak_mb = torch.cuda.max_memory_allocated(device) / (1024 ** 2)
            freeze_memory[strategy_name] = peak_mb
            print(f"Peak GPU memory: {peak_mb:.1f} MB")
        else:
            freeze_memory[strategy_name] = float("nan")
        final_iou = hist["test_iou"][-1]
        if final_iou > freeze_best_iou:
            freeze_best_iou = final_iou
            freeze_best_weights = model.state_dict()
            freeze_best_strategy = strategy_name
    print(f"\nBest strategy: {freeze_best_strategy} with Test IoU =␣
↪{freeze_best_iou:.4f}")
    return freeze_histories, freeze_best_weights, freeze_best_iou,␣
↪freeze_memory, freeze_best_strategy


NUM_BLOCKS = 4
```

```
freeze_histories, freeze_best_weights, freeze_best_iou, freeze_memory,␣
↪freeze_best_strategy = (
    run_freeze_strategy_experiments(num_blocks=NUM_BLOCKS, epochs=5)
)
```

```
==================================================
Strategy: Full fine-tune (freeze 0)
==================================================
Encoder blocks frozen: 0/4 | Trainable params: 8,560,193 / 8,560,193


Epoch 1: Train Loss=0.2121, Test Loss=0.1550 | Test IoU=0.8333, Test Pixel
Acc=0.8886


Epoch 2: Train Loss=0.1492, Test Loss=0.1368 | Test IoU=0.8499, Test Pixel
Acc=0.9065


Epoch 3: Train Loss=0.1380, Test Loss=0.1215 | Test IoU=0.8652, Test Pixel
Acc=0.9133


Epoch 4: Train Loss=0.1250, Test Loss=0.1213 | Test IoU=0.8662, Test Pixel
Acc=0.9180


Epoch 5: Train Loss=0.1152, Test Loss=0.1142 | Test IoU=0.8757, Test Pixel
Acc=0.9232
Peak GPU memory: 4336.3 MB

==================================================
Strategy: Freeze 1 block
==================================================
Encoder blocks frozen: 1/4 | Trainable params: 8,521,473 / 8,560,193


Epoch 1: Train Loss=0.2044, Test Loss=0.1663 | Test IoU=0.8270, Test Pixel
Acc=0.8813


Epoch 2: Train Loss=0.1461, Test Loss=0.1283 | Test IoU=0.8588, Test Pixel
Acc=0.9092


Epoch 3: Train Loss=0.1344, Test Loss=0.1221 | Test IoU=0.8674, Test Pixel
Acc=0.9173
```

```
Epoch 4: Train Loss=0.1176, Test Loss=0.1096 | Test IoU=0.8798, Test Pixel
Acc=0.9243


Epoch 5: Train Loss=0.1101, Test Loss=0.1130 | Test IoU=0.8727, Test Pixel
Acc=0.9230
Peak GPU memory: 3995.3 MB

==================================================
Strategy: Freeze 2 blocks
==================================================
Encoder blocks frozen: 2/4 | Trainable params: 8,300,033 / 8,560,193


Epoch 1: Train Loss=0.1907, Test Loss=0.1520 | Test IoU=0.8301, Test Pixel
Acc=0.8941


Epoch 2: Train Loss=0.1357, Test Loss=0.1654 | Test IoU=0.8236, Test Pixel
Acc=0.8928


Epoch 3: Train Loss=0.1216, Test Loss=0.1101 | Test IoU=0.8800, Test Pixel
Acc=0.9267


Epoch 4: Train Loss=0.1099, Test Loss=0.0921 | Test IoU=0.8963, Test Pixel
Acc=0.9370


Epoch 5: Train Loss=0.1021, Test Loss=0.1027 | Test IoU=0.8866, Test Pixel
Acc=0.9302
Peak GPU memory: 3840.0 MB

==================================================
Strategy: Freeze all encoder (4 blocks)
==================================================
Encoder blocks frozen: 4/4 | Trainable params: 3,874,817 / 8,560,193


Epoch 1: Train Loss=0.1733, Test Loss=0.1134 | Test IoU=0.8770, Test Pixel
Acc=0.9227


Epoch 2: Train Loss=0.1259, Test Loss=0.1051 | Test IoU=0.8826, Test Pixel
Acc=0.9258
```

```
Epoch 3: Train Loss=0.1173, Test Loss=0.1057 | Test IoU=0.8833, Test Pixel
Acc=0.9268


Epoch 4: Train Loss=0.1077, Test Loss=0.0891 | Test IoU=0.8983, Test Pixel
Acc=0.9370


Epoch 5: Train Loss=0.0993, Test Loss=0.0926 | Test IoU=0.9000, Test Pixel
Acc=0.9380
Peak GPU memory: 3701.3 MB

Best strategy: Freeze all encoder (4 blocks) with Test IoU = 0.9000
```

[35]:
```python
def save_safwan_freeze_artifacts(
    freeze_best_weights,
    freeze_best_strategy,
    freeze_best_iou,
    freeze_histories,
    freeze_memory,
    checkpoint_dir=None,
):
    """Save best regularized U-Net checkpoint, histories, and peak-memory JSON.

    Args:
        freeze_best_weights (dict | None): State dict for best run.
        freeze_best_strategy (str | None): Strategy label.
        freeze_best_iou (float): Best IoU achieved.
        freeze_histories (dict): Per-strategy training curves.
        freeze_memory (dict): Peak GPU MB per strategy (NaN on CPU).
        checkpoint_dir (pathlib.Path | None): Output directory.

    Returns:
        pathlib.Path: Directory used.
    """
    if checkpoint_dir is None:
        checkpoint_dir = Path.cwd() / "checkpoints"
    checkpoint_dir.mkdir(parents=True, exist_ok=True)
    if freeze_best_weights is not None:
        torch.save(
            {
                "model_state_dict": freeze_best_weights,
                "strategy": freeze_best_strategy,
                "iou": float(freeze_best_iou),
```

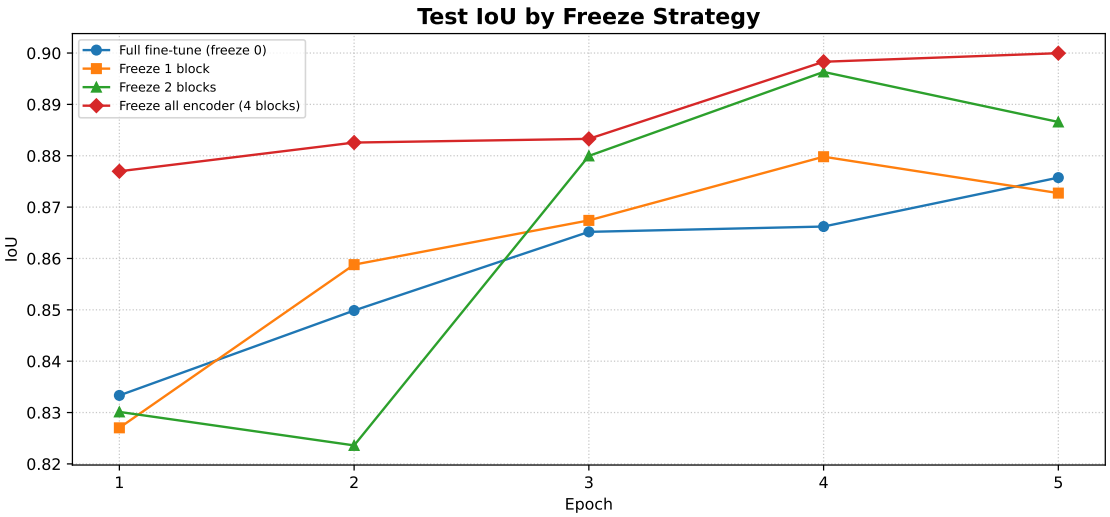
```
            },
            checkpoint_dir / "safwan_best_unet_regularized.pt",
        )
        print(
            f"Saved: safwan_best_unet_regularized.pt  ({freeze_best_strategy},␣
↪IoU={freeze_best_iou:.4f})"
        )
    else:
        print("No freeze_best_weights found; run the Safwan training cell first.
↪")
    if freeze_histories:
        with open(checkpoint_dir / "safwan_freeze_histories.json", "w") as f:
            json.dump(freeze_histories, f, indent=2)
        print("Saved: safwan_freeze_histories.json")
    if freeze_memory:
        mem_out = {k: (None if isinstance(v, float) and v != v else v) for k, v␣
↪in freeze_memory.items()}
        with open(checkpoint_dir / "safwan_freeze_memory.json", "w") as f:
            json.dump(mem_out, f, indent=2)
        print("Saved: safwan_freeze_memory.json")
    return checkpoint_dir


CHECKPOINT_DIR = save_safwan_freeze_artifacts(
    freeze_best_weights,
    freeze_best_strategy,
    freeze_best_iou,
    freeze_histories,
    freeze_memory,
)
```

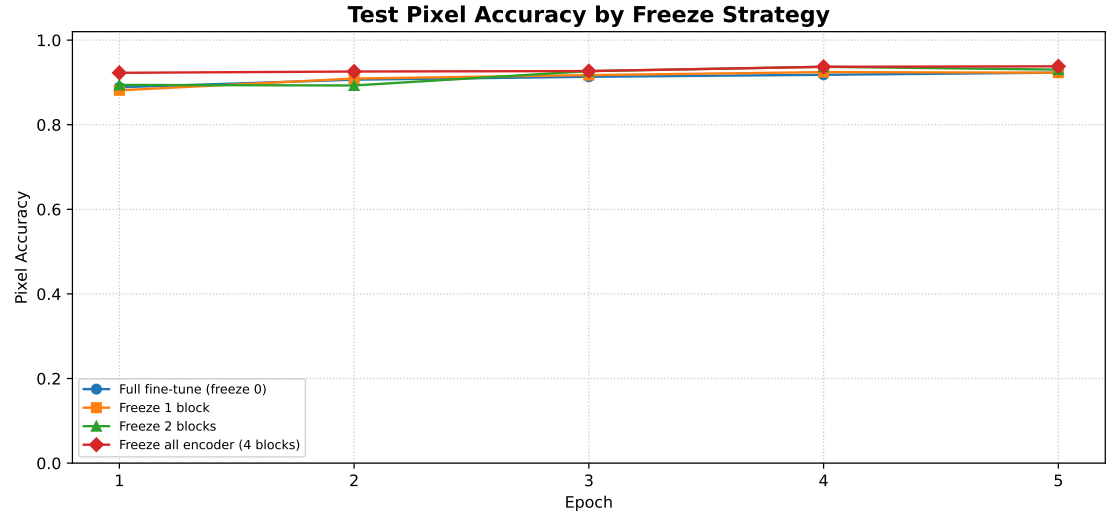
```
Saved: safwan_best_unet_regularized.pt  (Freeze all encoder (4 blocks),
IoU=0.9000)
Saved: safwan_freeze_histories.json
Saved: safwan_freeze_memory.json
```

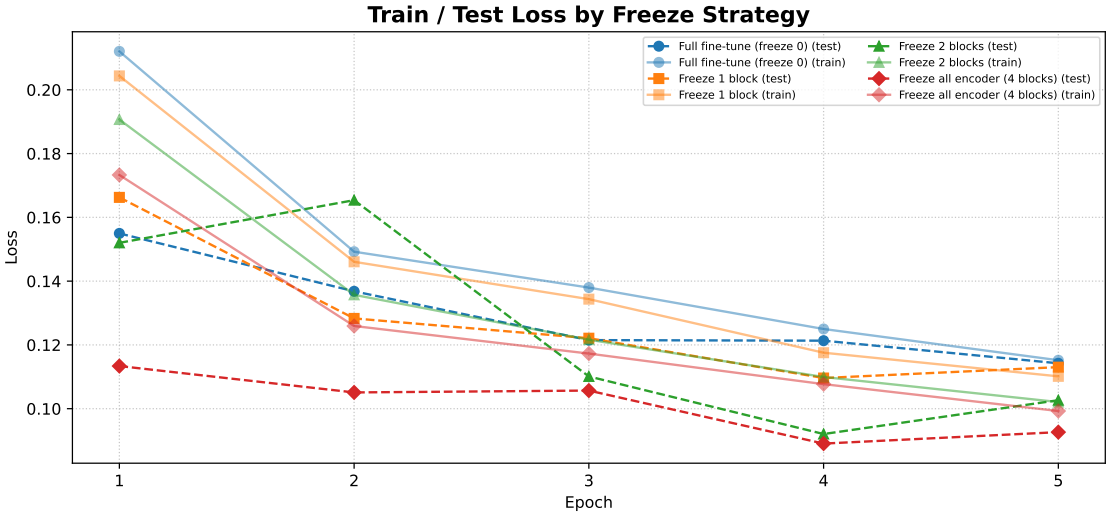
2.1.4 Results: comparison across freeze strategies

[36]:
```
def plot_freeze_strategy_training_curves(freeze_histories):
    """Plot IoU, pixel accuracy, and train/test loss for each freeze strategy.

    Args:
        freeze_histories (dict): Maps strategy name -> history dict from␣
↪``train_and_evaluate``.

    Returns:
        None
```

```
    """
    epochs_range = list(range(1, 6))
    fig, axes = plt.subplots(3, 1, figsize=(10, 14))
    markers = ["o", "s", "^", "D"]
    colors = ["tab:blue", "tab:orange", "tab:green", "tab:red"]
    for idx, (name, hist) in enumerate(freeze_histories.items()):
        m, c = markers[idx], colors[idx]
        axes[0].plot(epochs_range, hist["test_iou"], marker=m, color=c,␣
↪label=name)
        axes[1].plot(epochs_range, hist["test_acc"], marker=m, color=c,␣
↪label=name)
        axes[2].plot(epochs_range, hist["test_loss"], marker=m, color=c,␣
↪linestyle="--", label=f"{name} (test)")
        axes[2].plot(epochs_range, hist["train_loss"], marker=m, color=c,␣
↪linestyle="-", alpha=0.5, label=f"{name} (train)")
    axes[0].set_title("Test IoU by Freeze Strategy", fontsize=14,␣
↪fontweight="bold")
    axes[0].set_ylabel("IoU")
    axes[0].set_xlabel("Epoch")
    axes[0].set_xticks(epochs_range)
    axes[0].grid(True, linestyle=":", alpha=0.7)
    axes[0].legend(fontsize=8)
    axes[1].set_title("Test Pixel Accuracy by Freeze Strategy", fontsize=14,␣
↪fontweight="bold")
    axes[1].set_ylabel("Pixel Accuracy")
    axes[1].set_xlabel("Epoch")
    axes[1].set_xticks(epochs_range)
    axes[1].set_ylim(0, 1.02)
    axes[1].grid(True, linestyle=":", alpha=0.7)
    axes[1].legend(fontsize=8)
    axes[2].set_title("Train / Test Loss by Freeze Strategy", fontsize=14,␣
↪fontweight="bold")
    axes[2].set_ylabel("Loss")
    axes[2].set_xlabel("Epoch")
    axes[2].set_xticks(epochs_range)
    axes[2].grid(True, linestyle=":", alpha=0.7)
    axes[2].legend(fontsize=7, ncol=2)
    plt.tight_layout()
    plt.show()


plot_freeze_strategy_training_curves(freeze_histories)
```


Test IoU by Freeze Strategy
Full fine-tune (freeze 0)
Freeze 1 block
Freeze 2 blocks
Freeze all encoder (4 blocks)
IoU
Epoch
0.90
0.89
0.88
0.87
0.86
0.85
0.84
0.83
0.82
1
2
3
4
5


Test Pixel Accuracy by Freeze Strategy
Full fine-tune (freeze 0)
Freeze 1 block
Freeze 2 blocks
Freeze all encoder (4 blocks)
Pixel Accuracy
Epoch
1.0
0.8
0.6
0.4
0.2
0.0
1
2
3
4
5


Train / Test Loss by Freeze Strategy
Full fine-tune (freeze 0) (test)
Full fine-tune (freeze 0) (train)
Freeze 1 block (test)
Freeze 1 block (train)
Freeze 2 blocks (test)
Freeze 2 blocks (train)
Freeze all encoder (4 blocks) (test)
Freeze all encoder (4 blocks) (train)
Loss
Epoch
0.20
0.18
0.16
0.14
0.12
0.10
1
2
3
4
5

2.1.5 GPU / Memory & Parameter Summary

[37]:
```
def print_freeze_strategy_summary_table(freeze_histories, freeze_memory,␣
↪device):
    """Print final IoU, accuracy, and peak memory per strategy.

    Args:
        freeze_histories (dict): Strategy -> history.
        freeze_memory (dict): Strategy -> peak MB (may be NaN).
        device (torch.device): Device used for training.

    Returns:
        None
    """
    print("=" * 65)
    print(f"{'Strategy':<35} {'Final IoU':>10} {'Final Acc':>10} {'Peak MB':
↪>8}")
    print("=" * 65)
    for name, hist in freeze_histories.items():
        iou = hist["test_iou"][-1]
        acc = hist["test_acc"][-1]
        mem = freeze_memory.get(name, float("nan"))
        mem_str = f"{mem:.1f}" if not (mem != mem) else "CPU"
        print(f"{name:<35} {iou:>10.4f} {acc:>10.4f} {mem_str:>8}")
    print("=" * 65)
    print(f"\nDevice used: {device}")
    if torch.cuda.is_available():
        print(f"GPU: {torch.cuda.get_device_name(device)}")
        total_mem = torch.cuda.get_device_properties(device).total_memory /␣
↪(1024**2)
        print(f"Total GPU memory: {total_mem:.0f} MB")
    else:
        print("Training ran on CPU.")


print_freeze_strategy_summary_table(freeze_histories, freeze_memory, device)
```

```
=================================================================
Strategy                              Final IoU  Final Acc  Peak MB
=================================================================
Full fine-tune (freeze 0)                0.8757     0.9232   4336.3
Freeze 1 block                           0.8727     0.9230   3995.3
Freeze 2 blocks                          0.8866     0.9302   3840.0
Freeze all encoder (4 blocks)            0.9000     0.9380   3701.3
=================================================================

Device used: cuda
GPU: NVIDIA GeForce RTX 4060 Laptop GPU
```

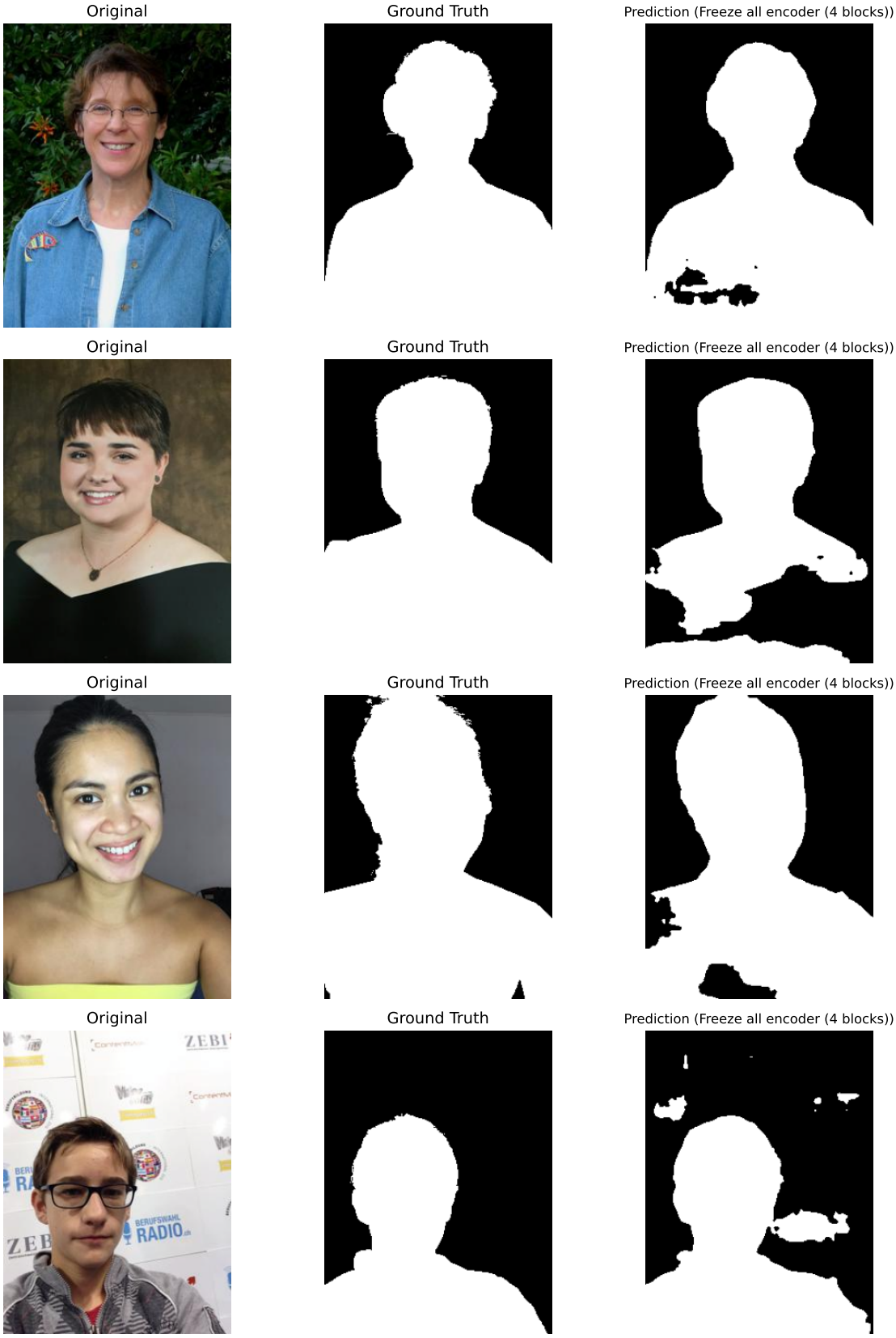
```
Total GPU memory: 8188 MB
```

2.1.6 Prediction samples: best freeze strategy

[38]:
```python
def visualize_best_regularized_predictions(
    freeze_best_weights,
    freeze_best_strategy,
    freeze_best_iou,
    num_blocks: int,
    test_loader,
    device,
):
    """Load best frozen encoder weights and plot a batch of predictions.

    Args:
        freeze_best_weights (dict): State dict.
        freeze_best_strategy (str): Winning strategy name.
        freeze_best_iou (float): Best IoU (for title).
        num_blocks (int): Encoder depth for model construction.
        test_loader (DataLoader): Test data.
        device (torch.device): Device.

    Returns:
        UNetRegularized: Model in eval mode.
    """
    model = UNetRegularized(num_classes=1, num_blocks=num_blocks).to(device)
    model.load_state_dict(freeze_best_weights)
    model.eval()
    images, true_masks = next(iter(test_loader))
    images = images.to(device)
    with torch.no_grad():
        logits = model(images)
        preds = (torch.sigmoid(logits) > 0.5).float()
    images_np = images.cpu().permute(0, 2, 3, 1).numpy()
    true_masks_np = true_masks.cpu().squeeze(1).numpy()
    preds_np = preds.cpu().squeeze(1).numpy()
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    num_show = min(4, images_np.shape[0])
    fig, axes = plt.subplots(num_show, 3, figsize=(12, 4 * num_show))
    for i in range(num_show):
        img = np.clip(images_np[i] * std + mean, 0, 1)
        axes[i, 0].imshow(img)
        axes[i, 0].set_title("Original", fontsize=13)
        axes[i, 0].axis("off")
        axes[i, 1].imshow(true_masks_np[i], cmap="gray")
        axes[i, 1].set_title("Ground Truth", fontsize=13)
```

```
        axes[i, 1].axis("off")
        axes[i, 2].imshow(preds_np[i], cmap="gray")
        axes[i, 2].set_title(f"Prediction ({freeze_best_strategy})",␣
↪fontsize=11)
        axes[i, 2].axis("off")
    plt.suptitle(
        f"Best freeze strategy: {freeze_best_strategy} | Test IoU =␣
↪{freeze_best_iou:.4f}",
        fontsize=14,
        fontweight="bold",
        y=1.01,
    )
    plt.tight_layout()
    plt.show()
    return model


best_freeze_model = visualize_best_regularized_predictions(
    freeze_best_weights,
    freeze_best_strategy,
    freeze_best_iou,
    NUM_BLOCKS,
    test_loader,
    device,
)
```

Best freeze strategy: Freeze all encoder (4 blocks) | Test IoU = 0.9000



2.1.7 Interpretation & Lessons I Learned

1. **Transfer learning depth matters.** The number of frozen encoder blocks directly affects convergence speed and final IoU. Freezing low-level feature blocks (edges, textures) usually preserves useful ImageNet representations without hurting segmentation quality.

2. **Regularized decoder.** Adding BatchNorm and Dropout to the decoder blocks (which Khoroshilova's original U-Net did not have) helps stabilize training, especially important when only a fraction of parameters are being updated (frozen-encoder scenarios).

3. **Practical GPU benefits.** Freezing encoder blocks reduces the number of gradient tensors stored during backpropagation, lowering peak GPU memory. This is valuable when training on consumer GPUs with limited VRAM.

Next steps: - **Gradual unfreezing**: train decoder-only for a few epochs, then progressively unfreeze encoder blocks with a lower learning rate. - **Learning rate scheduling**: combine freeze strategies with cosine annealing or step decay. - **More epochs**: 5 epochs may not fully reveal the gap between strategies; 15-20 epochs would give clearer separation.

Demo can be found in demo.ipynb